Centro Universitário de Excelência
Sistemas de Informação

OAT 2 - MecâniQA João Pessoa

Autor: Arthur de Aquino Anjos
Henrique Estrela Santos
Luan Vinicius Miranda da Silva
Matheus Kaick Rocha da Fonseca
Rafael Pinto Germano Pereira

Feira de Santana, 2026

Agenda

Como os cuidados de boas práticas, a validação temporal e os novos atributos elevaram a confiabilidade das previsões de demanda da MecaniQA.

1. Feature Engineering Temporal
Criação de lags e janelas rolantes causais a partir da série de trocas de óleo

2. Prevenção de Vazamento de Dados
Cuidados documentados no código contra uso de dados futuros

3. Validação Cruzada Temporal
Refatoração da divisão treino e teste com TimeSeriesSplit

4. Motor de Avaliação
Cálculo de MAE, RMSE e MAPE para os baselines de referência

5. Considerações Finais
Resultados dos baselines e desempenho na validação temporal

Contexto e Objetivo desta Etapa

- Esta etapa dá continuidade à análise da MecaniQA, com a base já tratada
- O foco passou a ser a confiabilidade da avaliação dos baselines, não só as previsões
- Foram adicionados atributos temporais e uma validação que respeita a ordem cronológica
- Os cuidados contra vazamento de dados e as boas práticas de avaliação ficaram documentados no código

Resultado: Novos Atributos Temporais

- Lag_1, Lag_7 e Lag_30: valor da série em dias anteriores, para o modelo enxergar padrões de curto e longo prazo
- Rolling_Mean_7 e Rolling_Mean_30: média móvel usando dias passados, suavizando picos isolados
- dados_com_features reúne esses atributos, prontos para alimentar modelos supervisionados futuros
- QA aprovado: nenhuma coluna usa o dia atual ou dados futuros, evitando vazamento

	Trocas_Oleo	Trocas_Oleo_Lag_1	Trocas_Oleo_Lag_7	Trocas_Oleo_Lag_30	Trocas_Oleo_Rolling_Mean_7	Trocas_Oleo_Rolling_Mean_30
2025-12-27	44.00	43.00	42.00	24.00	33.43	31.70
2025-12-28	45.00	44.00	43.00	34.00	33.71	32.37
2025-12-29	27.00	45.00	23.00	41.00	34.00	32.73
2025-12-30	27.00	27.00	24.00	38.00	34.57	32.27
2025-12-31	30.00	27.00	30.00	22.00	35.00	31.90

OBS: Mesma lógica replicada para Manutencao_Motor (colunas omitidas por espaço)

Código: Criação dos Lags e Janelas Rolantes

- A função ffill() preenche lacunas repetindo o último valor conhecido, sem utilizar informações futuras
- O comando shift(n) cria os lags deslocando a série para frente, para hoje nunca aparecer na previsão
- rolling().mean().shift(1) cria as janelas rolantes sem olhar o dia atual, com shift adicional após a média
- Os asserts finais comparam cada coluna à fórmula original, travando o notebook se algo vazar

```
dados_temp['Trocas_Oleo'] = dados_temp['Trocas_Oleo'].ffill()
dados_temp['Manutencao_Motor'] = dados_temp['Manutencao_Motor'].ffill()

# Criar Lags para 'Trocas_Oleo'
dados_temp['Trocas_Oleo_Lag_1'] = dados_temp['Trocas_Oleo'].shift(1)
dados_temp['Trocas_Oleo_Lag_7'] = dados_temp['Trocas_Oleo'].shift(7)
dados_temp['Trocas_Oleo_Lag_30'] = dados_temp['Trocas_Oleo'].shift(30)

dados_temp['Trocas_Oleo_Rolling_Mean_7'] = dados_temp['Trocas_Oleo'].rolling(window=7).mean().shift(1)
dados_temp['Trocas_Oleo_Rolling_Mean_30'] = dados_temp['Trocas_Oleo'].rolling(window=30).mean().shift(1)
```

OBS: Mesma lógica replicada para Manutencao_Motor (códigos omitidos por espaço)

Quais cuidados foram adotados para garantir uma avaliação correta?

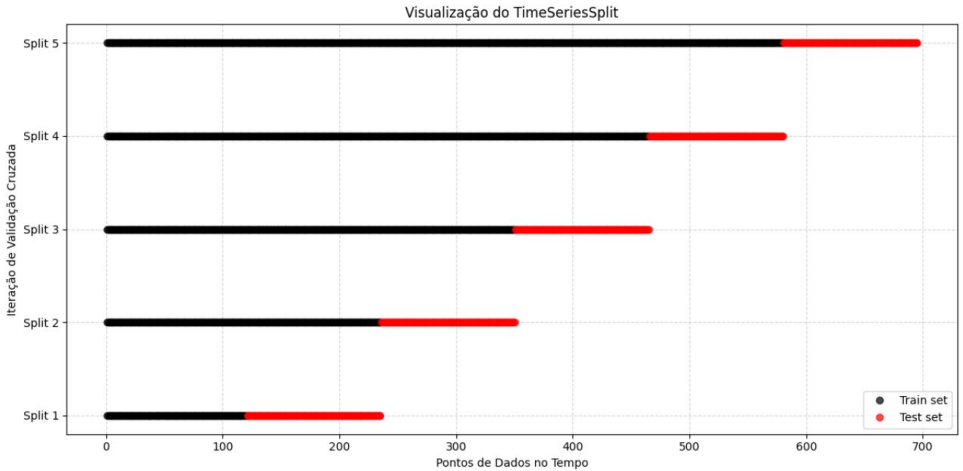
- As variáveis temporais foram construídas somente com informações disponíveis antes do momento da previsão
- A validação respeita a ordem cronológica, mantendo os períodos de teste posteriores aos períodos de treino
- O TimeSeriesSplit evita o embaralhamento dos dados e simula uma situação mais próxima do uso real
- As verificações de QA funcionam como uma camada de segurança para identificar possíveis usos indevidos de dados

```
# Asserts de QA para garantir que não haja vazamento de dados
# O primeiro valor não-NaN de Lag_1 deve ser o primeiro valor da
assert dados_com_features['Trocas_Oleo_Lag_1'].iloc[0] == dados_t
assert dados_com_features['Trocas_Oleo_Lag_7'].iloc[0] == dados_t
assert dados_com_features['Trocas_Oleo_Lag_30'].iloc[0] == dados_
```

OBS: Print mostra apenas o início dos asserts de QA, a verificação completa é mais extensa no notebook

Resultado: Validação Cruzada Temporal

- TimeSeriesSplit(n_splits=5): cinco divisões cronológicas em vez de uma divisão aleatória
- Treino cresce a cada divisão e o teste é sempre o período seguinte, imitando o uso real
- Essa simulação evita avaliar o modelo com dados que ainda não existiriam na prática
- O mesmo esquema foi aplicado aos três baselines, para uma comparação justa



Código: TimeSeriesSplit e Motor de Avaliação

- O método avaliar_modelo() usa TimeSeriesSplit para repetir o treino e o teste cinco vezes
- Em cada divisão, o baseline Naive usa o último valor do treino, enquanto as médias móveis usam a janela correspondente
- A função metricas_erro() calcula MAE, RMSE e MAPE em cada divisão
- O resultado final é a média das cinco divisões, permitindo uma avaliação mais consistente que um único teste

```python
def metricas_erro(y_true, y_pred):
    # Remove NaNs antes de calcular as métricas para
    # Isso garante que apenas os pares de valores (re
    valid_indices = y_true.notna() & y_pred.notna()
    y_true_clean = y_true[valid_indices]
    y_pred_clean = y_pred[valid_indices]

def avaliar_modelo(serie, n_splits=5):
    tscv = TimeSeriesSplit(n_splits=n_splits)

    maes = []
    rmses = []
    mapes = []

    for train_index, test_index in tscv.split(serie):
        train = serie.iloc[train_index]
        test = serie.iloc[test_index]
```

OBS: Print mostra apenas a declaração inicial das funções, a execução completa está no notebook

Resultados: Métricas de Desempenho

- MAE, erro médio absoluto: a MM30 apresentou o menor erro, com 7,34, seguida pela MM7 (7,37) e pelo Naive (9,23)
- RMSE, que penaliza mais os erros grandes: a MM7 apresentou 8,10, abaixo da MM30 (8,18) e do Naive (10,99)
- MAPE, erro percentual: a MM7 teve 37,51%, muito próxima da MM30 (37,56%) e abaixo do Naive (50,75%)
- As duas médias móveis superaram o Naive nas três métricas, com resultados próximos entre MM7 e MM30

```
Avaliando baselines para Trocas_Oleo com TimeSeriesSplit:

Métricas Médias (Trocas de Óleo):
MAE:
 Naive    9.231304
MM7       7.365714
MM30      7.339362
dtype: float64

RMSE:
 Naive    10.990522
MM7        8.104645
MM30       8.176437
dtype: float64

MAPE:
 Naive    50.749692
MM7       37.511667
MM30      37.562234
dtype: float64
```

OBS: Resultados mostrados apenas para Trocas de Óleo, o mesmo cálculo foi feito para Manutenção de Motor

Considerações finais

- Os atributos de lag e rolling window foram criados de forma estritamente causal, para evitar vazamento de dados nas previsões
- A validação passou a usar TimeSeriesSplit, eliminando a mistura entre passado e futuro
- As médias móveis superaram o Naive nas três métricas, com a MM30 apresentando o menor MAE e a MM7 os menores RMSE e MAPE
- Os cuidados contra vazamento de dados ficaram documentados no código, com QA automático

Referências

- FILHO, Mario. ***Como prever séries temporais univariadas usando Python***. Disponível em: https://mariofilho.com/como-prever-series-temporais-univariadas-usando-python/.
- DATACAMP. ***Análise e decomposição de séries temporais em Python***. Disponível em: https://www.datacamp.com/pt/tutorial/time-series-decomposition.
- NERD DOS DADOS. ***Como fazer SÉRIES TEMPORAIS com Python***. YouTube. Disponível em: https://www.youtube.com/watch?v=yfVEymFbJSo
- HOWELL, Egor. **Cross Validation for Time Series Forecasting | Python Tutorial**. YouTube. Disponível em: https://www.youtube.com/watch?v=1rZpbvSI26c
- LET'S DATA. **Séries temporais com Python**. YouTube. Disponível em: https://youtu.be/1m9u10eJ3Mk